sharing data between Angular components using an Angular service

---

Why Use a Service for Component Communication?

In Angular, **components should not directly depend on each other** unless they have a strict parent–child relationship.
When components are:

- Distant in the component tree
- Unrelated (loaded via routing)

the **recommended approach** is to use a **shared service**.

Key Benefits

- Centralized state management
- Loose coupling between components
- Reusable and testable logic
- Works across routed components

High-Level Architecture

Conceptually:

- A **service** acts as a **single source of truth**
- Components **publish data** to the service
- Other components **subscribe** to changes

Communication Patterns Using Services

1. Simple Shared Property (Not Reactive)

2. Observable-Based Communication (Recommended)

3. BehaviorSubject for State Sharing (Most Common)

We will focus on **best practice: BehaviorSubject + Observable**.

Step-by-Step Implementation

Step 1: Create a Shared Service

ng generate service services/data-sharing

data-sharing.service.ts

```
import { Injectable } from '@angular/core';
import { BehaviorSubject } from 'rxjs';

@Injectable({
  providedIn: 'root'
})
export class DataSharingService {

  private messageSource = new BehaviorSubject<string>('Initial Message');

  message$ = this.messageSource.asObservable();

  updateMessage(newMessage: string): void {
    this.messageSource.next(newMessage);
  }
}
```

Why BehaviorSubject?

- Holds the **latest value**
- New subscribers immediately receive data
- Ideal for application state

Step 2: Sender Component (Publish Data)

sender.component.ts

```
import { Component } from '@angular/core';
import { DataSharingService } from '../services/data-sharing.service';

@Component({
  selector: 'app-sender',
  template: `
    <h3>Sender Component</h3>
    <input type="text" #msg />
    <button (click)="send(msg.value)">Send</button>
  `
})
export class SenderComponent {

  constructor(private dataService: DataSharingService) {}

  send(value: string): void {
    this.dataService.updateMessage(value);
  }
}
```

Step 3: Receiver Component (Consume Data)

receiver.component.ts

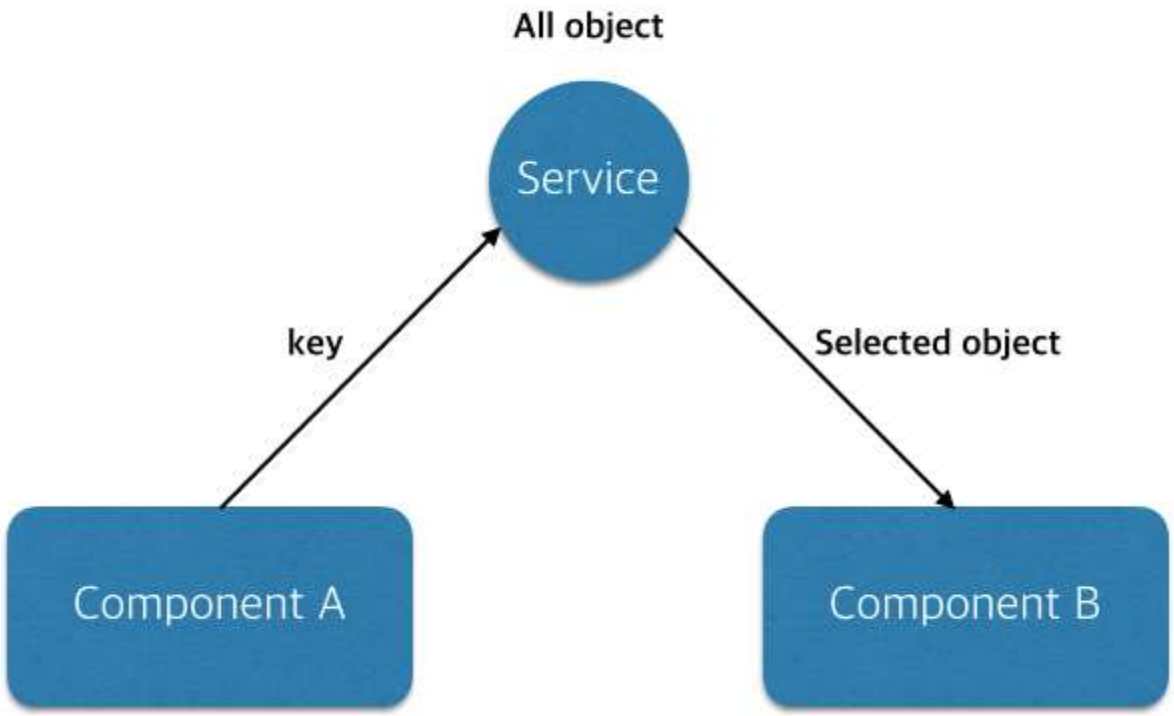
```
import { Component, OnInit } from '@angular/core';
import { DataSharingService } from '../services/data-sharing.service';
```

```
@Component({
  selector: 'app-receiver',
  template: `
    <h3>Receiver Component</h3>
    <p>Received Message: {{ message }}</p>
  `
})
export class ReceiverComponent implements OnInit {

  message!: string;

  constructor(private dataService: DataSharingService) {}

  ngOnInit(): void {
    this.dataService.message$.subscribe(value => {
      this.message = value;
    });
  }
}
```

Step 4: Works Across Routing

This approach **works even if components are loaded via Angular Router**, because:

- Service is singleton (providedIn: 'root')
- State persists across navigation



Common Real-World Use Cases

Use Case	Example
Authentication State	Logged-in user info
Cart Management	Shopping cart items
Theme / Language	Dark mode, locale
Cross-Component Events	Refresh notifications
Dashboard Filters	Shared filter criteria

Comparison with Other Communication Techniques

Technique	Use Case	Limitation
@Input / @Output	Parent–Child	Not scalable
ViewChild	Tight coupling	Fragile
Service + Observable	Any component	Best practice
NgRx / Signals	Large apps	Higher complexity
